Unit 1

Python Character Set

In Python, a character set refers to the collection of valid characters that can be used in the language to write programs. These characters include letters, digits, symbols, and white-space.

1. Letters:

Uppercase letters: A – Z

Lowercase letters: a – z

Python is case-sensitive.

Example: Name and name are different identifiers.

2. Digits:

0 – 9

Digits are used in numeric literals (integers, floats, etc.) and in identifiers (but identifiers cannot begin with a digit).

3. Special Characters / Symbols:

Symbols are used for operations, punctuation, grouping, etc.

Examples: + - * / % = < > ( ) [ ] { } , : . ' " # @

4. Whitespace Characters:

Whitespace characters help separate tokens in a program. Examples:

Space ( )

Tab (\t)

Newline (\n)

Carriage return (\r)

Python uses indentation (whitespace) to define code blocks instead of braces { }.

5. Other Characters:

Underscore (_): used in identifiers (e.g., my_variable).

Escape Sequences: backslash (\) followed by a character to represent special meaning.

Examples:

\n → newline

\t → tab

\\ → backslash itself

Python Tokens

In Python, a token is the smallest unit in a program that has meaning to the compiler/interpreter.

Types of Python Tokens

1. Keywords:

Keywords are reserved words with predefined meanings. It cannot be used as variable names.

Examples:

if, else, while, for, break, continue, return, def, class, import, True, False, None

2. Identifiers:

Identifiers are the names given to variables, functions, classes, etc.

Rules:

- Can contain letters, digits, and underscore (_)
- Cannot start with a digit

Case-sensitive

Cannot be a keyword

Example:

name = "Ram"

Age = 18

3. Literals:

Literals are the constant values assigned to variables.

Types:

Numeric Literals: 10, 3.14, 0b101, 0x1F

String Literals: "hello", 'world', '"multi-line"'

Boolean Literals: True, False

Special Literal: None

Example:

```
a = 100            # Integer literal
b = 5.46          # Float literal
dept = "CSM"        # String literal
is_valid = True   #Boolean
score = None   #None
```

4. Punctuators (Separators):

Characters that structure the program.

Examples:

```
def add(a, b):
    return a + b
```

5. Comments:

Not executed by Python, used for documentation.

Single-line: # comment

Multi-line: enclosed in triple quotes (''' ... ''' or """ ... """)

Example:

```
# This is a single-line comment
"""
This is a
multi-line comment
"""
```

Operators in Python

In Python, operators are special symbols that perform operations on variables and values.

Types of Operators in Python

1. Arithmetic Operators

Used for basic mathematical operations.

Operator	Meaning	Example (`a=10, b=3`)	Result
`+`	Addition	`a + b`	13
`-`	Subtraction	`a - b`	7
`*`	Multiplication	` a * b`	30
`/`	Division (float)	`a / b`	3.33
`%`	Modulus (remainder)	`a % b`	1
`//`	Floor Division	`a // b`	3

`**`	Exponentiation	`a ** b`	1000

2. Relational (Comparison) Operators

Used to compare values, returns True/False.

Operator	Meaning	Example (`a=10, b=3`)	Result
`==`	Equal to	`a == b`	False
`!=`	Not equal to	`a != b`	True
`>`	Greater than	`a > b`	True
`<`	Less than	`a < b`	False
`>=`	Greater or equal	`a >= b`	True
`<=`	Less or equal	`a <= b`	False

3. Logical Operators

Used to combine conditional statements.

Operator	Meaning	Example (`a=10, b=3`)	Result
`and`	True if both are True	`(a > 5 and b > 1)`	True
`or`	True if at least one is True	`(a > 5 or b > 20)`	True
`not`	Negates condition	`not(a > 5)`	False

4. Assignment Operators

Used to assign values (with or without operations).

Operator	Example (`a=10`)	Equivalent to
`=`	`a = 10`	`a = 10`
`+=`	`a += 5`	`a = a + 5`
`-=`	`a -= 3`	`a = a - 3`
`*=`	`a *= 2`	`a = a * 2`

`/=`	`a /= 2`	`a = a / 2`
`%=`	`a %= 3`	`a = a % 3`
`//=`	`a //= 2`	`a = a // 2`
`**=`	`a **= 3`	`a = a ** 3`

5. Bitwise Operators

Work on bits (0s and 1s).

Operator	Meaning	Example (`a=6(110), b=3(011)`)	Result
`&`	Bitwise AND	`a & b` → `110 & 011`	`010 (2)`
|	Bitwise OR	\`a b`→`110 | 011\`	`111 (7)`
`^`	Bitwise XOR	`a ^ b` → `110 ^ 011`	`101 (5)`
`~`	Bitwise NOT	`~a` → `-(a+1)`	`-7`
`<<`	Left Shift	`a << 1` → `1100`	`12`
`>>`	Right Shift	`a >> 1` → `011`	`3`

6. Membership Operators

Used to check membership in sequences (like list, string, tuple).

Operator	Meaning	Example	Result
`in`	True if value exists	`'a' in 'apple'`	True
`not in`	True if value does not exist	`'x' not in 'apple'`	True

7. Identity Operators

Used to compare memory locations of objects.

Operator	Meaning	Example
`is`	True if both refer to same object	`a is b`
`is not`	True if both refer to different objects	`a is not b`

Example:

```
x = [1, 2, 3]

y = x

z = [1, 2, 3]

print(x is y)        # True   (same object)

print(x is z)        # False (different objects with same content)
```

Input and Output in Python

In Python, Input and Output operations are used to take data from the user and display results to the user.

Input in Python:

- Python uses the input() function to take input from the user.
- It always returns data as a string (you may need to convert it).

Example 1: Taking string input

```
name = input("Enter your name: ")

print("Hello,", name)
```

Output:

Hello, Ashu

Example 2: Taking integer input

```
age = int(input("Enter your age: "))

print("Next year you will be", age + 1)
```

Example 3: Taking multiple inputs

```
a, b = map(int, input("Enter two numbers: ").split())

print("Sum =", a + b)
```

Output in Python

Python uses the print() function for output.

Example 1: Simple output

```
print("Hello World")
```

Example 2: Printing variables

```
x = 10

y = 20

print("x =", x, "y =", y)
```

Output: x = 10 y = 20

Example 3: Using sep and end parameters

```
print("Apple", "Banana", "Cherry", sep=", ")

print("Hello", end=" ")

print("World")
```

Output:

Apple, Banana, Cherry

Hello World

Example 4: Formatted output (f-strings)

```
name = "Ashu"

age = 20

print(f"My name is {name} and I am {age} years old.")
```

Program Control Flow

In Python, Program Control Flow means the order in which instructions are executed. Python controls flow using conditional statements, loops, and jump statements.

1. Conditional (Decision-Making) Statements

Used when you want to execute certain parts of code based on conditions.

a) if statement

```
x = 10
if x > 0:
    print("Positive number")
```

b) if–else statement

```
x = -3
if x >= 0:
    print("Positive or Zero")
else:
    print("Negative number")
```

c)if-elif-else ladder

```
marks = 75
if marks >= 90:
    print("Grade A")
elif marks >= 60:
    print("Grade B")
else:
    print("Grade C")
```

d)Nested if

```
x = 15
if x > 0:
```

```
    if x % 2 == 0:
        print("Positive Even")
    else:
        print("Positive Odd")
```

3. Looping (Iteration Statements)

Used to repeat a block of code multiple times.

a) while loop

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

b) for loop

```
for i in range(1, 6):
    print(i)
```

c) Nested loops

```
for i in range(3):
    for j in range(2):
        print(i, j)
```

4. Jumping Statements

Used to alter normal flow inside loops.

a) break → exits the loop completely

```
for i in range(1, 6):
    if i == 3:
```

```
        break
    print(i)
```

b) continue → skips current iteration

```
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

c) pass → does nothing (placeholder)

```
for i in range(3):
    pass    # used to write code later
```

Functions in Python

A function is a block of reusable code that performs a specific task. Instead of writing the same code multiple times, you can put it inside a function and call it whenever needed.

Types of Functions in Python

a) Built-in functions → Already provided by Python.

Examples: print(), len(), type(), max(), min(), range().

```
print(len("Python"))    # 6
```

b) User-defined functions → Created by the programmer.

```
def greet():
    print("Hello, Python!")
greet()
```

Creating a Function

General syntax:

```
def function_name(parameters):
    """Optional docstring"""
    # function body
    return value
```

Function with Parameters

Functions can accept arguments (inputs).

```
def add(a, b):
    return a + b
print(add(5, 3))     # 8
```

Types of Arguments

a) Positional Arguments

```
def power(base, exp):
    return base ** exp
print(power(2, 3))     # 8
```

b) Keyword Arguments

```
print(power(exp=4, base=3))     # 81
```

c) Default Arguments

```
def greet(name="User"):
    print("Hello,", name)
greet()                 # Hello, User
greet("Ashu")         # Hello, Ashu
```

d) Variable-length Arguments

i)*args → Non-keyword arguments (tuple)

```
def total(*numbers):
    return sum(numbers)
print(total(1, 2, 3, 4))     # 10
```

ii)**kwargs → Keyword arguments (dictionary)

```
def info(**details):
    for key, value in details.items():
        print(key, ":", value)


info(name="Ashu", age=21, city="Hyderabad")
```

e) Return Statement

Functions can return values using return.

```
def square(x):
    return x * x
result = square(6)
print(result)     # 36
```

Modules in Python

A module in Python is simply a file that contains Python code (functions, classes, variables).

A Python file (.py) is a module.

You can use/import functions, variables, and classes from one file into another.

Example:

Suppose you have a file math_utils.py

```
def add(a, b):
    return a + b


def subtract(a, b):
    return a - b
```

Now, in another file:

```
import math_utils


print(math_utils.add(5, 3))          # 8
print(math_utils.subtract(10, 4)) # 6
```

Types of Modules

a) Built-in Modules → Already provided with Python.

Examples: math, os, sys, datetime, random.

```
import math
print(math.sqrt(16))      # 4.0
```

b) User-defined Modules → Created by the programmer (like math_utils.py)

c) Third-party Modules → Installed using pip.

Examples: numpy, pandas, matplotlib.

```
import numpy as np
arr = np.array([1, 2, 3])
print(arr)
```

3. Importing Modules

Different ways to import:

a) import module

```
import math
print(math.pi)
```

b) import module as alias

```
import math as m
print(m.pi)
```

c) from module import function

```
from math import sqrt, pi
print(sqrt(25))     # 5.0
print(pi)           # 3.14159...
```

d) from module import *

```
from math import *
print(sin(0))     # 0.0
```

4. The dir() Function

Lists all functions/variables inside a module.

```
import math
print(dir(math))
```

5. The __name__ Variable

Every Python module has a special variable __name__.

If the file is run directly → __name__ == "__main__".

Helps differentiate between running a file directly vs importing it.

Example:

```
# mymodule.py
def greet():
    print("Hello from module")
if __name__ == "__main__":
    greet()
```

Packages in Python

A package is a way of organizing related modules into a single directory.

A module = single .py file.

A package = folder (directory) that contains multiple modules and a special file __init__.py.

a) Structure of a Package

Example folder structure:

```
mypackage/
    __init__.py
    math_utils.py
    string_utils.py
```

b) Creating a Package

Step 1: Create the folder mypackage/

Step 2: Create module files inside it.

math_utils.py

```
def add(a, b):
    return a + b
```

string_utils.py

```
def shout(text):
    return text.upper()
```

init.py

```
from .math_utils import add
from .string_utils import shout
```

Importing a Package

In another Python file:

```
import mypackage
print(mypackage.add(5, 3))          # 8
print(mypackage.shout("hello"))     # HELLO
```

Or import specific modules:

```
from mypackage import math_utils, string_utils
print(math_utils.add(10, 20))           # 30
print(string_utils.shout("python"))     # PYTHON
```

c) Nested Packages

A package can also contain sub-packages:

```
mypackage/
    __init__.py
    math/
        __init__.py
        operations.py
    strings/
        __init__.py
```

formatters.py

Then import like:

from mypackage.math import operations

from mypackage.strings import formatters

Object-Oriented Programming in Python

OOP is a programming paradigm that organizes code into objects (real-world entities) that have attributes (data) and methods (behavior).

Python is a fully object-oriented language.

1. Key Concepts of OOP

a) Class

A blueprint for creating objects.

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color
    def drive(self):
        print(f"{self.color} {self.brand} is driving...")
```

b) Object

An instance of a class.

```
car1 = Car("Toyota", "Red")
car2 = Car("BMW", "Black")
car1.drive()    # Red Toyota is driving...
```

```
car2.drive()     # Black BMW is driving...
```

c) The __init__ Method (Constructor)

Special method that runs automatically when an object is created.

Used to initialize attributes.

```
class Student:
    def __init__(self, name, roll):
        self.name = name
        self.roll = roll
```

d) Attributes and Methods

Attributes → variables inside a class.

Methods → functions inside a class.

```
class Student:
    def __init__(self, name, roll):
        self.name = name     # attribute
        self.roll = roll
    def show(self):            # method
        print(self.name, self.roll)
s1 = Student("Ashu", 101)
s1.show()
```

2. OOP Principles in Python

a) Encapsulation (Data Hiding)

Keeping data safe by using private/protected variables.

```
class Bank:
```

```
    def __init__(self, balance):
        self.__balance = balance     # private variable
    def deposit(self, amount):
        self.__balance += amount
    def get_balance(self):
        return self.__balance
```

b) Inheritance (Reusing Code)

One class can inherit properties/methods from another.

```
class Animal:
    def speak(self):
        print("Animal speaks")
class Dog(Animal):     # Dog inherits Animal
    def speak(self):
        print("Bark")
dog = Dog()
dog.speak()     # Bark
```

c) Polymorphism (Many Forms)

Same method name → different behaviors.

```
class Bird:
    def sound(self):
        print("Chirp")
class Cat:
    def sound(self):
```

```
        print("Meow")
for obj in (Bird(), Cat()):
    obj.sound()
```

d) Abstraction (Hiding Implementation Details)

Defined using abstract base classes (abc module).

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
        return 3.14 * self.r * self.r
c = Circle(5)
print(c.area())     # 78.5
```

3. Special (Magic / Dunder) Methods

Python classes have built-in special methods.

Example:

__init__() → constructor

__str__() → string representation

__len__() → used by len()

```
class Book:
    def __init__(self, title):
        self.title = title
    def __str__(self):
        return f"Book: {self.title}"
b = Book("Python OOP")
print(b)     # Book: Python OOP
```

Files in Python

File handling in Python allows you to store data permanently and work with text or binary files.

We use the built-in open() function to open files and perform operations.

1. Opening a File

Syntax:

file = open("filename", mode)

filename → name of the file (e.g., "data.txt")

mode → tells how the file should be opened

Common Modes:

"r" → Read (default)

"w" → Write (creates new file or overwrites existing)

"a" → Append (adds content at end)

"b" → Binary mode (rb, wb)

"x" → Create (error if file exists)

"r+" → Read and write

Example:

```
f = open("data.txt", "r")
```

2. Reading from a File

```
f = open("data.txt", "r")
print(f.read())          # Read entire file
print(f.readline())      # Read one line
print(f.readlines())     # Read all lines into a list
f.close()
```

3. Writing to a File

```
f = open("data.txt", "w")     # overwrite if exists
f.write("Hello, Python!\n")
f.write("File handling is easy.")
f.close()
```

Append Mode:

```
f = open("data.txt", "a")
f.write("\nAdding new line.")
f.close()
```

4. Using with Statement

Automatically closes file after use.

```
with open("data.txt", "r") as f:
    content = f.read()
    print(content)
```

5. Working with Binary Files

```
with open("image.jpg", "rb") as f:
    data = f.read()
```

6. File Object Methods

Some useful methods:

f.read(size) → Read given number of characters/bytes

f.readline() → Read one line

f.readlines() → List of all lines

f.write(str) → Write a string

f.seek(offset) → Move cursor to given position

f.tell() → Current file position

f.close() → Close the file

7. Example Program

```
# Write to file
with open("notes.txt", "w") as f:
    f.write("Python File Handling\n")
    f.write("Second Line")
# Read from file
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

MS Excel in Python

Python provides powerful libraries to create, read, update, and analyze Excel files (.xlsx, .xls).

The most commonly used libraries are:

openpyxl → For .xlsx files (modern Excel format).

xlrd, xlwt → For older .xls files (less common now).

pandas → For data analysis with Excel files.

1. Reading Excel Files

Using openpyxl

```
import openpyxl

# Load Excel workbook
wb = openpyxl.load_workbook("data.xlsx")

# Select sheet
sheet = wb.active

# Read cell value
print(sheet["A1"].value)
```

```
# Iterate through rows
for row in sheet.iter_rows(values_only=True):
    print(row)
```

Using pandas

```
import pandas as pd

# Read Excel file into DataFrame
df = pd.read_excel("data.xlsx")

print(df.head())    # Show first 5 rows
```

2. Writing Excel Files

Using openpyxl

```
import openpyxl

wb = openpyxl.Workbook()    # Create new workbook
sheet = wb.active

sheet["A1"] = "Name"
sheet["B1"] = "Age"

sheet["A2"] = "Ashu"
```

```
sheet["B2"] = 21


wb.save("students.xlsx")
```

Using pandas

```
import pandas as pd
data = {
    "Name": ["Ashu", "Lovely", "Ravi"],
    "Marks": [90, 85, 95]
}
df = pd.DataFrame(data)
# Save to Excel
df.to_excel("marks.xlsx", index=False)
```

3. Updating Excel Files

```
import openpyxl
wb = openpyxl.load_workbook("students.xlsx")
sheet = wb.active
sheet["B2"] = 22     # Update age
wb.save("students_updated.xlsx")
```

4. Working with Multiple Sheets

```
import openpyxl
wb = openpyxl.Workbook()
wb.create_sheet("Marks")         # Add new sheet
```

```
wb.create_sheet("Attendance")
print(wb.sheetnames)              # List all sheets
```

5. Styling Excel Files (openpyxl)

```
from openpyxl import Workbook
from openpyxl.styles import Font
wb = Workbook()
sheet = wb.active
cell = sheet["A1"]
cell.value = "Hello Excel"
cell.font = Font(bold=True, color="FF0000")     # Bold, Red text
wb.save("styled.xlsx")
```

6. Useful Libraries

openpyxl → Reading/writing .xlsx files

pandas → Data analysis + Excel integration

xlsxwriter → Advanced Excel reports with charts

xlrd/xlwt → Older .xls files